МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа № 2 по курсу «Операционные системы и архитектура компьютера»

Выполнила: Д. В. Новикова
Группа: М8О-204БВ-25
Преподаватель: Н. А. Гапонов
Дата: 23.09.2026
Оценка: _____

Москва, 2026

Условие

Цель работы: освоить создание дочернего процесса и обмен данными между процессами через неименованные каналы в Linux.

Задание: разработать две программы на C. Родитель получает имя выходного файла и строки, передаёт их дочернему процессу через pipe1. Дочерний проверяет строки, записывает подходящие в файл, а сведения об ошибках передаёт родителю через pipe2. Обрабатывать ошибки системных вызовов.

Вариант: 15. Строка должна начинаться с заглавной буквы.

Метод решения

Программа состоит из процессов parent и child. Parent читает имя файла, создаёт два канала и вызывает fork(). В дочерней ветви pipe1 подключается к стандартному вводу, а записывающий конец pipe2 — к дескриптору 3; затем execv() запускает child с именем файла в аргументах. Parent передаёт все строки через pipe1, закрывает его после конца ввода, читает сообщения из pipe2 и ожидает завершения child. Child записывает строки с начальной латинской буквой A–Z в файл; остальные строки считает ошибочными и сообщает их номера родителю.

Описание программы

Исходные файлы находятся в src. Программа собирается CMake в две исполняемые программы. Проверка начального символа реализована отдельной функцией starts_with_upper(); строки произвольной длины читаются getline().

Структура файлов

Таблица 1 – Файлы проекта

Файл	Назначение
`src/parent.c`	Чтение ввода, создание каналов и процесса, передача строк, вывод ошибок и ожидание child.
`src/child.c`	Проверка строк, запись подходящих в файл, передача ошибок по дескриптору 3.
`src/CMakeLists.txt`	Настройка сборки двух программ на C99 с предупреждениями компилятора.

Основные типы данных и функции

Основные данные: pipe1[2] и pipe2[2] хранят концы каналов; pid_t pid — идентификатор дочернего процесса; ssize_t s_len — длину прочитанной строки. write_all() повторяет write() до отправки всех байтов и обрабатывает EINTR и EPIPE. starts_with_upper() возвращает истину, если первый байт строки находится в диапазоне 'A'–'Z'.

Используемые системные вызовы

Для управления процессами и обменом используются вызовы и интерфейсы POSIX из таблицы 2. Файловые функции fopen(), fileno(), getline(), dprintf() и fclose() применяются для работы с потоками и файлом.

Таблица 2 – Используемые системные вызовы

Системный вызов	Назначение в программе
`pipe()`	Создание двух неименованных каналов.
`fork()`	Создание дочернего процесса.
`dup2()`	Подключение канала к stdin и дескриптору 3; перенаправление stdout в файл.
execv()	Запуск отдельной программы child в дочернем процессе.
read(), write()	Чтение сообщений из pipe2 и передача строк через pipe1.
close()	Закрытие ненужных концов каналов и файловых дескрипторов.
waitpid()	Ожидание завершения дочернего процесса и получение его статуса.

Ключевые фрагменты кода

Родитель создаёт каналы, разделяет выполнение через fork() и подключает их к дочернему процессу. Проверка ошибок pipe(), fork() и dup2() предотвращает продолжение работы с недействительными дескрипторами.

Листинг 1 – Создание канала и дочернего процесса (parent.c)

```
int pipe1[2], pipe2[2];
if (pipe(pipe1) == -1) { perror("pipe1"); return EXIT_FAILURE; }
if (pipe(pipe2) == -1) { perror("pipe2"); return EXIT_FAILURE; }
pid_t pid = fork();
if (pid == -1) { perror("fork"); return EXIT_FAILURE; }
if (pid == 0) {
    close(pipe1[1]); close(pipe2[0]);
    if (dup2(pipe1[0], STDIN_FILENO) == -1) exit(EXIT_FAILURE);
    if (dup2(pipe2[1], CHILD_ERR_FD) == -1) exit(EXIT_FAILURE);
    char *argv[] = {"child", file_name, NULL};
    execv(CHILD_PATH, argv);
}
```

Результаты

Проверка выполнена в Ubuntu после сборки командой cmake -S src -B /tmp/os_lab2_build && cmake --build /tmp/os_lab2_build. На вход поданы имя result.txt и четыре строки: Alpha, beta, Gamma и пустая строка. Ниже приведён фактический вывод программы; затем показано содержимое созданного файла.

```
$ printf "result.txt\nAlpha\nbeta\nGamma\n\n" | ./parent
Line 2 does not start with an uppercase letter: beta
Line 4 does not start with an uppercase letter:
enter string to send to child process:
child process exited successfully
$ cat result.txt
Alpha
Gamma
```

Демонстрация работы strace

Трассировка снята при указанном запуске командой strace -f -o /tmp/os_lab2_strace.txt -e trace=execve,clone,pipe,pipe2,dup2,close,read,write,openat,wait4 ./parent. Ниже приведены вызовы, подтверждающие создание каналов, запуск child, передачу данных и ожидание завершения. Полный вывод этой трассировки находится в приложении Б.

Для каждой строки strace указывает PID процесса. В данном запуске PID родителя — 58177, дочернего процесса — 58178.

```
58177 pipe2([3, 4], 0) = 0
58177 pipe2([5, 6], 0) = 0
```

Созданы каналы pipe1 и pipe2. В первом дескриптор 3 предназначен для чтения, 4 — для записи; во втором соответственно 5 и 6.

```
58177 clone(...) = 58178
58178 execve("./child", ["child", "result.txt"], ...) = 0
```

После fork() дочерний процесс запускает child с именем файла. Далее parent передаёт строки через дескриптор 4, читает ошибки из дескриптора 5 и вызывает wait4(), соответствующий waitpid() в исходном коде.

Выводы

Разработаны две программы на C для взаимодействия через два неименованных канала. Проверочный запуск подтвердил запись строк Alpha и Gamma в файл и передачу сообщений о строках beta и пустой строке родителю. Трассировка показала создание каналов, запуск дочерней программы, обмен данными и ожидание её завершения. Проверка первого символа реализована для латинских заглавных букв A–Z.

Приложение А. Исходный код программы

Ниже приведён полный исходный код файлов проекта без сокращений.

Листинг А.1 – Файл parent.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <errno.h>
#include <signal.h>

#define CHILD_PATH "./child"
#define CHILD_ERR_FD 3

static int write_all(int fd, const char *buf, size_t len) {
    size_t written = 0;
    while (written < len) {
        ssize_t n = write(fd, buf + written, len - written);
        if (n == -1) {
            if (errno == EINTR) {
                continue;
            }
            if (errno == EPIPE) {
                fprintf(stderr, "error: child closed the pipe, cannot send data\n");
            } else {
                perror("write to pipe1");
            }
            return -1;
        }
        written += (size_t)n;
    }
    return 0;
}

int main(void)
{

    signal(SIGPIPE, SIG_IGN);

    char *file_name = NULL;
    size_t cap = 0;
    ssize_t s_len = getline(&file_name, &cap, stdin);

    if (s_len <=0) {
        fprintf(stderr, "error: reading input\n");
        return EXIT_FAILURE;
    }

    if (file_name[s_len - 1] == '\n') {
        file_name[s_len - 1] = '\0';
    }
    if (file_name[0] == '\0') {
        fprintf(stderr, "error: empty file name\n");
        free(file_name);
        return EXIT_FAILURE;
    }

    int pipe1[2];
    int pipe2[2];
```

```
    if (pipe(pipe1) == -1) {
        perror("pipe1");
        free(file_name);
        return EXIT_FAILURE;
    }

    if (pipe(pipe2) == -1) {
        perror("pipe2");
        free(file_name);
        close(pipe1[0]);
        close(pipe1[1]);
        return EXIT_FAILURE;
    }

    pid_t pid = fork();
    if (pid == -1) {
        perror("fork");
        free(file_name);
        close(pipe1[0]);
        close(pipe1[1]);
        close(pipe2[0]);
        close(pipe2[1]);
        return EXIT_FAILURE;
    }

    if (pid == 0) {
        //child process
        close(pipe1[1]);
        close(pipe2[0]);
        if (dup2(pipe1[0], STDIN_FILENO) == -1) {
            perror("child process, dup2");
            exit(EXIT_FAILURE);
        }
        close(pipe1[0]);

        if (dup2(pipe2[1], CHILD_ERR_FD) == -1) {
            perror("child process, dup2");
            exit(EXIT_FAILURE);
        }

        close(pipe2[1]);

        char *argv[] = {"child", file_name, NULL};
        execv(CHILD_PATH, argv);
        perror("execv");
        exit(EXIT_FAILURE);

    } else {
        //parent process
        close(pipe1[0]);
        close(pipe2[1]);

        printf("enter string to send to child process:\n");

        char *line = NULL;
        size_t cap = 0;
        ssize_t s_len;

        while ((s_len = getline(&line, &cap, stdin)) != -1) {
```

```
            if (write_all(pipe1[1], line, (size_t)s_len) == -1) {
                break;
            }
        }

        if (ferror(stdin)) {
            perror("getline");
        }

        free(line);
        close(pipe1[1]);

        char buffer[4096];
        ssize_t bytes_read;

        while (1) {
            bytes_read = read(pipe2[0], buffer, sizeof(buffer));

            if (bytes_read == -1) {
                if (errno == EINTR) {
                    continue;
                }
                perror("read from pipe2");
                break;
            }

            if (bytes_read == 0) {
                break;
            }

            if (write_all(STDOUT_FILENO, buffer, bytes_read) == -1) {
                break;
            }
        }

        close(pipe2[0]);

        int status;
        if (waitpid(pid, &status, 0) == -1) {
            perror("waitpid");
            free(file_name);
            return EXIT_FAILURE;
        }

        if (WIFEXITED(status) && WEXITSTATUS(status) == 0) {
            printf("child process exited successfully\n");
        } else if (WIFEXITED(status)){
            printf("child process exited with status %d\n", WEXITSTATUS(status));
        } else{
            printf("child process exited abnormally\n");
        }

        free(file_name);
        return EXIT_SUCCESS;
    }
}
```

Листинг А.2 – Файл child.c

```
#include <stdio.h>
```

```c
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

#define CHILD_ERR_FD 3

int starts_with_upper(const char *str) {
    if (str == NULL || str[0] == '\0') {
        return 0;
    }
    return (str[0] >= 'A' && str[0] <= 'Z');
}

int main(int argc, char *argv[]) {
    if (argc < 2) {
        dprintf(CHILD_ERR_FD, "Filename was not provided\n");
        return EXIT_FAILURE;
    }

    const char *file_name = argv[1];
    FILE *file = fopen(file_name, "w");
    if (!file) {
        dprintf(CHILD_ERR_FD, "Failed to open file: %s\n", file_name);
        return EXIT_FAILURE;
    }

    int file_descriptor = fileno(file);
    if (file_descriptor == -1) {
        perror("fileno");
        fclose(file);
        return EXIT_FAILURE;
    }

    if (dup2(file_descriptor, STDOUT_FILENO) == -1) {
        perror("dup2");
        fclose(file);
        return EXIT_FAILURE;
    }
    close(file_descriptor);

    char *line = NULL;
    size_t cap = 0;
    ssize_t s_len;
    size_t s_read = 0;

    while ((s_len = getline(&line, &cap, stdin)) != -1) {
        s_read ++;
        if (s_len > 0 && line[s_len - 1] == '\n') {
            line[s_len - 1] = '\0';
        }

        if (starts_with_upper(line)) {
            printf("%s\n", line);
        } else {
            dprintf(CHILD_ERR_FD, "Line %zu does not start with an uppercase letter:
%s\n", s_read, line);
        }
    }
```

```
    free(line);
    fclose(stdout);
    exit(EXIT_SUCCESS);
}
```

Листинг А.3 – Файл CMakeLists.txt

```
cmake_minimum_required(VERSION 3.10)

project(2nd_lab, LANGUAGES C)

set(CMAKE_C_STANDARD 99)
add_compile_options(-Wall -Wextra)

add_executable(parent parent.c)
add_executable(child child.c)
```

Приложение Б. Полный вывод strace

Полный вывод выбранных для трассировки системных вызовов, полученный командой, указанной в основном тексте. Строки не сокращены и приведены в порядке записи strace.

Листинг Б.1 – Вывод strace -f с фильтром системных вызовов

```
58177 execve("./parent", ["./parent"], 0x7fff321625e8 /* 25 vars */) = 0
58177 openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
58177 close(3)                          = 0
58177 openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
58177 read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\220\243\2\0\0\0\0\0"...,
832) = 832
58177 close(3)                          = 0
58177 read(0, "result.txt\nAlpha\nbeta\nGamma\n\n", 4096) = 29
58177 pipe2([3, 4], 0)                  = 0
58177 pipe2([5, 6], 0)                  = 0
58177 clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD,
child_tidptr=0x701746e54a10) = 58178
58177 close(3)                          = 0
58177 close(6 <unfinished ...>
58178 close(4 <unfinished ...>
58177 <... close resumed>)              = 0
58178 <... close resumed>)              = 0
58178 close(5)                          = 0
58177 write(4, "Alpha\n", 6 <unfinished ...>
58178 dup2(3, 0 <unfinished ...>
58177 <... write resumed>)              = 6
58178 <... dup2 resumed>)               = 0
58177 write(4, "beta\n", 5 <unfinished ...>
58178 close(3 <unfinished ...>
58177 <... write resumed>)              = 5
58178 <... close resumed>)              = 0
58177 write(4, "Gamma\n", 6 <unfinished ...>
58178 dup2(6, 3 <unfinished ...>
58177 <... write resumed>)              = 6
58178 <... dup2 resumed>)               = 3
58177 write(4, "\n", 1 <unfinished ...>
58178 close(6 <unfinished ...>
58177 <... write resumed>)              = 1
58178 <... close resumed>)              = 0
58177 read(0,  <unfinished ...>
58178 execve("./child", ["child", "result.txt"], 0x7ffc7b1f3d38 /* 25 vars */
<unfinished ...>
58177 <... read resumed>"", 4096)       = 0
58177 close(4)                          = 0
58177 read(5,  <unfinished ...>
58178 <... execve resumed>)             = 0
58178 openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 4
58178 close(4)                          = 0
58178 openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 4
58178 read(4, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\220\243\2\0\0\0\0\0"...,
832) = 832
58178 close(4)                          = 0
58178 openat(AT_FDCWD, "result.txt", O_WRONLY|O_CREAT|O_TRUNC, 0666) = 4
58178 dup2(4, 1)                        = 1
58178 close(4)                          = 0
58178 read(0, "Alpha\nbeta\nGamma\n\n", 4096) = 18
58178 write(3, "Line 2 does not start with an up"..., 53 <unfinished ...>
58177 <... read resumed>"Line 2 does not start with an up"..., 4096) = 53
```

```
58178 <... write resumed>)              = 53
58177 write(1, "Line 2 does not start with an up"..., 53 <unfinished ...>
58178 write(3, "Line 4 does not start with an up"..., 49 <unfinished ...>
58177 <... write resumed>)              = 53
58178 <... write resumed>)              = 49
58177 read(5,  <unfinished ...>
58178 read(0,  <unfinished ...>
58177 <... read resumed>"Line 4 does not start with an up"..., 4096) = 49
58178 <... read resumed>"", 4096)       = 0
58177 write(1, "Line 4 does not start with an up"..., 49 <unfinished ...>
58178 write(1, "Alpha\nGamma\n", 12 <unfinished ...>
58177 <... write resumed>)              = 49
58178 <... write resumed>)              = 12
58177 read(5,  <unfinished ...>
58178 close(1)                          = 0
58177 <... read resumed>"", 4096)       = 0
58178 +++ exited with 0 +++
58177 --- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=58178, si_uid=1000,
si_status=0, si_utime=0, si_stime=0} ---
58177 close(5)                          = 0
58177 wait4(58178, [{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 58178
58177 write(1, "enter string to send to child pr"..., 73) = 73
58177 +++ exited with 0 +++
```